

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной инженерии и Компьютерной техники

Лабораторная работа №6
Численное решение обыкновенных
дифференциальных уравнений
Вариант 5

Выполнила:
Косогова Мария Александровна
Группа: Р3206
Проверил:
Зинченко Антон Андреевич,
Преподаватель практики.

Санкт-Петербург 2026

Содержание

ЗАДАНИЕ	3
ОСНОВНЫЕ ЭТАПЫ ВЫПОЛНЕНИЯ.....	6
ВЫВОД.....	16

Задание

- Методы: Усовершенствованный метод Эйлера, Метод Рунге-Кутты 4-го порядка, Адамса

1. В программе численные методы решения обыкновенных дифференциальных уравнений (ОДУ) должен быть реализован в виде отдельного класса /метода/функции;
2. Пользователь выбирает ОДУ вида $y = f(x, y)$ (не менее трех уравнений), из тех, которые предлагает программа;
3. Предусмотреть ввод исходных данных с клавиатуры: начальные условия $y_0 = y(x_0)$, интервал дифференцирования $[x_0, x_n]$, шаг h , точность ε ;
4. Для исследования использовать одношаговые методы и многошаговые методы (см. табл.1);
5. Составить таблицу приближенных значений интеграла дифференциального уравнения, удовлетворяющего начальным условиям, для всех методов, реализуемых в программе;
6. Для оценки точности одношаговых методов использовать правило Рунге;
7. Для оценки точности многошаговых методов использовать точное решение задачи: $\varepsilon = \max_{0 \leq i \leq n} |y_{i\text{точн}} - y_i|$
8. Построить графики точного решения и полученного приближенного решения (разными цветами);
9. Программа должна быть протестирована при различных наборах данных, в том числе и некорректных.
10. Проанализировать результаты работы программы.

Формулы

- Усовершенствованный метод Эйлера (порядок точности $p = 2$)

- Итерационная формула:

$$y_{i+1} = y_i + \frac{h}{2} [k_1 + k_2], \quad \text{где}$$

$$k_1 = f(x_i, y_i)$$

$$k_2 = f(x_i + h, y_i + h \cdot k_1)$$

- Геометрический смысл: использование среднего значения производной на отрезке $[x_i, x_{i+1}]$

- Метод Рунге-Кутты 4-го порядка (порядок точности $p = 4$)

- Итерационная формула:

$$y_{i+1} = y_i + \frac{1}{6} (k_1 + 2k_2 + 2k_3 + k_4), \quad \text{где}$$

$$k_1 = h \cdot f(x_i, y_i)$$

$$k_2 = h \cdot f\left(x_i + \frac{h}{2}, y_i + h \cdot \frac{k_1}{2}\right)$$

$$k_3 = h \cdot f\left(x_i + \frac{h}{2}, y_i + h \cdot \frac{k_2}{2}\right)$$

$$k_4 = h \cdot f(x_i + h, y_i + k_3)$$

- Метод Адамса (предиктор-корректор, 4-й порядок)

- Этап 1: первые 4 значения y_0, y_1, y_2, y_3 вычисляются методом Рунге-Кутты 4-го порядка

- Этап 2. Предиктор

$$y_{i+1}^{(p)} = y_i + \frac{h}{24} [55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3}]$$

- Этап 3. Корректор

$$y_{i+1}^{(c)} = y_i + \frac{h}{24} [9f(x_{i+1}, y_{i+1}^{(p)}) + 19f_i - 5f_{i-1} - f_{i-2}]$$

- Этап 4. Итерационное уточнение (при необходимости)

$$y_{i+1}^{(new)} = y_i + \frac{h}{24} [9f(x_{i+1}, y_{i+1}^{(c)}) + 19f_i - 5f_{i-1} - f_{i-2}]$$

Итерации продолжаются до выполнения условия: $|y_{i+1}^{(new)} - y_{i+1}^{(c)}| < \varepsilon$

- Оценка погрешности

- Правило Рунге (для одношаговых методов)

$$R = \frac{\max_{0 \leq i \leq n} |y_i^{(h)} - y_i^{(h/2)}|}{2^p - 1}$$

Критерий остановки: $R \leq \varepsilon$

- Оценка через точное решение (для многошаговых методов)

$$\varepsilon_{\text{факт}} = \max_{0 \leq i \leq n} |y_i^{\text{точн}} - y_i|$$

Основные этапы выполнения

- **Цель:** решить задачу Коши для обыкновенных дифференциальных уравнений численными методами.
- **Описание алгоритма решения задачи**
 1. Ввод исходных данных:
 - Выбор ОДУ из предложенных ($f(x,y)$, точное решение, начальные условия)
 - Интервал дифференцирования $[x_0, x_n]$
 - Шаг интегрирования h
 - Требуемая точность ε
 2. Проверка корректности входных данных:
 - $x_n > x_0, h > 0, h < (x_n - x_0), \varepsilon > 0$
 3. Для одношаговых методов (Эйлер, РК4):
 - а) Вычисление решения с шагом h
 - б) Вычисление решения с шагом $h/2$ для оценки погрешности
 - в) Применение правила Рунге: $R = \max|y_h - y_{h/2}| / (2^p - 1)$
 - г) Сравнение R с заданной точностью ε
 4. Для многошагового метода Адамса:
 - а) "Запуск" метода: вычисление первых 4 точек методом Рунге-Кутты 4-го порядка
 - б) Предиктор (прогноз): явная формула Адамса
 - в) Корректор (уточнение): неявная формула Адамса с итерациями до сходимости
 - г) Оценка погрешности через точное решение: $\varepsilon = \max|y_{\text{точн}} - y_i|$
 5. Формирование таблицы результатов для всех методов
 6. Построение графиков: точное решение и численные приближения
 7. Анализ погрешностей и сравнение методов

■ Листинг программы

```
import numpy as np
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
from tabulate import tabulate
import os

EQUATIONS = {
    1: {
        "label" : "y' = -2y + 2x2 + 2x, y(0)=1",
        "f" : lambda x, y: -2*y + 2*x**2 + 2*x,
        "exact" : lambda x: x**2 + np.exp(-2*x),
        "x0" : 0.0,
        "y0" : 1.0,
        "x_end" : 2.0,
        "h_def" : 0.1,
        "verify" : "y = x2 + e-2x",
    },
    2: {
        "label" : "y' = y - x2, y(0)=1",
        "f" : lambda x, y: y - x**2,
        "exact" : lambda x: x**2 + 2*x + 2 - np.exp(x),
        "x0" : 0.0,
        "y0" : 1.0,
        "x_end" : 1.5,
        "h_def" : 0.1,
        "verify" : "y = x2 + 2x + 2 - ex",
    },
    3: {
        "label" : "y' = -y + sin(x), y(0)=0.5",
        "f" : lambda x, y: -y + np.sin(x),
        "exact" : lambda x: 0.5*(np.sin(x) - np.cos(x)) +
np.exp(-x),
        "x0" : 0.0,
        "y0" : 0.5,
        "x_end" : 3.0,
        "h_def" : 0.2,
        "verify" : "y = (sin x - cos x)/2 + e-x",
    },
}

def euler_step(f, x, y, h):
    k1 = f(x, y)
    k2 = f(x + h, y + h * k1)
    return y + h / 2.0 * (k1 + k2)

def euler_solve(f, x0, y0, x_end, h):
    xs = [x0]
    ys = [y0]
    x, y = x0, y0
    n = int(round((x_end - x0) / h))
    for _ in range(n):
        y = euler_step(f, x, y, h)
        x = round(x + h, 14)
        xs.append(x)
        ys.append(y)
    return np.array(xs), np.array(ys)

def euler_runge_error(f, x0, y0, x_end, h, eps):
    p = 2
```

```

_, ys_h = euler_solve(f, x0, y0, x_end, h)
xs2, ys_h2 = euler_solve(f, x0, y0, x_end, h / 2)
ys_h2_at_h = ys_h2[::2]
R = np.max(np.abs(ys_h - ys_h2_at_h)) / (2**p - 1)
return R, xs2, ys_h2

def rk4_step(f, x, y, h):
    k1 = h * f(x, y)
    k2 = h * f(x + h/2, y + k1/2)
    k3 = h * f(x + h/2, y + k2/2)
    k4 = h * f(x + h, y + k3)
    return y + (k1 + 2*k2 + 2*k3 + k4) / 6.0

def rk4_solve(f, x0, y0, x_end, h):
    xs = [x0]
    ys = [y0]
    x, y = x0, y0
    n = int(round((x_end - x0) / h))
    for _ in range(n):
        y = rk4_step(f, x, y, h)
        x = round(x + h, 14)
        xs.append(x)
        ys.append(y)
    return np.array(xs), np.array(ys)

def rk4_runge_error(f, x0, y0, x_end, h, eps):
    p = 4
    _, ys_h = rk4_solve(f, x0, y0, x_end, h)
    xs2, ys_h2 = rk4_solve(f, x0, y0, x_end, h / 2)
    ys_h2_at_h = ys_h2[::2]
    min_len = min(len(ys_h), len(ys_h2_at_h))
    R = np.max(np.abs(ys_h[:min_len] -
ys_h2_at_h[:min_len])) / (2**p - 1)
    return R, xs2, ys_h2

def adams_solve(f, x0, y0, x_end, h, eps=1e-6):
    n = int(round((x_end - x0) / h))
    xs = np.array([x0 + i * h for i in range(n + 1)])
    ys = np.zeros(n + 1)
    fs = np.zeros(n + 1)

    ys[0] = y0
    fs[0] = f(xs[0], ys[0])
    for i in range(min(3, n)):
        ys[i + 1] = rk4_step(f, xs[i], ys[i], h)
        fs[i + 1] = f(xs[i + 1], ys[i + 1])

    for i in range(3, n):
        y_pred = ys[i] + h/24 * (55*fs[i] - 59*fs[i-1] +
37*fs[i-2] - 9*fs[i-3])
        f_pred = f(xs[i + 1], y_pred)

        y_corr = ys[i] + h/24 * (9*f_pred + 19*fs[i] -
5*fs[i-1] + fs[i-2])
        for _ in range(50):
            f_corr = f(xs[i + 1], y_corr)
            y_new = ys[i] + h/24 * (9*f_corr + 19*fs[i] -
5*fs[i-1] + fs[i-2])
            if abs(y_new - y_corr) < eps:
                y_corr = y_new

```



```

        break
        y_corr = y_new
        ys[i + 1] = y_corr
        fs[i + 1] = f(xs[i + 1], ys[i + 1])

    return xs, ys

def adams_max_error(xs, ys, exact):
    return np.max(np.abs(exact(xs) - ys))

def validate_inputs(x0, x_end, h, eps):
    errors = []
    if x_end <= x0:
        errors.append(" x_end должен быть больше x0")
    if h <= 0:
        errors.append(" Шаг h должен быть положительным")
    if h >= (x_end - x0):
        errors.append(" Шаг h слишком большой (больше интервала)")
    if eps <= 0:
        errors.append(" Точность ε должна быть положительной")
    return errors

def print_table(xs, ys_euler, ys_rk4, ys_adams, exact_func, header_extra=""):
    y_exact = exact_func(xs)
    rows = []
    for i in range(len(xs)):
        rows.append([
            i,
            f"{xs[i]:.4f}",
            f"{ys_euler[i]:.8f}" if i < len(ys_euler) else
            "-",
            f"{ys_rk4[i]:.8f}" if i < len(ys_rk4) else
            "-",
            f"{ys_adams[i]:.8f}" if i < len(ys_adams) else
            "-",
            f"{y_exact[i]:.8f}",
        ])
    headers = ["i", "x_i", "Эйлер улучш.", "Рунге-Кутта 4", "Адамс", "Точное"]
    print(header_extra)
    print(tabulate(rows, headers=headers, tablefmt="grid"))

def print_errors(xs, ys_euler, ys_rk4, ys_adams, exact_func, r_euler, r_rk4, eps):
    y_exact = exact_func(xs)
    err_euler = np.max(np.abs(y_exact - ys_euler))
    err_rk4 = np.max(np.abs(y_exact - ys_rk4))
    err_adams = np.max(np.abs(y_exact - ys_adams))

    print("\n" + "="*60)
    print(" ОЦЕНКА ПОГРЕШНОСТИ")
    print("="*60)
    print(f"\n Усовершенствованный метод Эйлера (p=2):")
    print(f" Правило Рунге: R = {r_euler:.2e} (ε задана = {eps:.2e})")
    print(f" {'✓ Точность достигнута' if r_euler <= eps else 'X Точность НЕ достигнута'}")

```

```

print(f"    Реальная погрешность: {err_euler:.2e}")

print(f"\n    Метод Рунге-Кутта 4-го порядка (p=4):")
print(f"    Правило Рунге: R = {r_rk4:.2e} (ε задана =
{eps:.2e})")
print(f"    {'✓ Точность достигнута' if r_rk4 <= eps
else 'X Точность НЕ достигнута'}")
print(f"    Реальная погрешность: {err_rk4:.2e}")

print(f"\n    Метод Адамса (предиктор-корректор, p=4):")
print(f"    ε = max|y_точн - y_i| = {err_adams:.2e}")

print("\n" + "-"*60)
print(f"    Сравнение методов по точности:")
print(f"    {'Метод':<35} {'max|δ|':>12}")
print(f"    {'-'*47}")
for name, err in [
    ("Усовершенствованный Эйлер", err_euler),
    ("Рунге-Кутта 4", err_rk4),
    ("Адамс", err_adams),
]:
    print(f"    {name:<35} {err:>12.2e}")
print("="*60 + "\n")

def plot_results(xs, ys_euler, ys_rk4, ys_adams, exact_func,
eq_label, filename):
    xs_fine = np.linspace(xs[0], xs[-1], 500)
    y_fine = exact_func(xs_fine)

    fig, ax = plt.subplots(figsize=(12, 7))
    fig.patch.set_facecolor('#1e1e2e')
    ax.set_facecolor('#181825')

    COLORS = {
        "exact": "#a6e3a1",
        "euler": "#f38ba8",
        "rk4": "#89b4fa",
        "adams": "#fab387",
        "grid": "#45475a",
        "fg": "#cdd6f4",
        "bg": "#1e1e2e",
    }

    ax.set_title(f"Численное решение ОДУ: {eq_label}",
        color=COLORS["fg"], fontsize=13, pad=15)
    ax.set_xlabel("x", color=COLORS["fg"], fontsize=11)
    ax.set_ylabel("y", color=COLORS["fg"], fontsize=11)
    ax.tick_params(colors=COLORS["fg"])
    for spine in ax.spines.values():
        spine.set_edgecolor(COLORS["grid"])
    ax.grid(True, color=COLORS["grid"], linewidth=0.6,
linestyle="--", alpha=0.7)

    ax.plot(xs_fine, y_fine, color=COLORS["exact"], lw=2.5,
        label="Точное решение", zorder=5)
    ax.plot(xs, ys_euler, 'o--', color=COLORS["euler"],
lw=1.2,
        markersize=4, label="Усовершенств. Эйлер",
zorder=4)
    ax.plot(xs, ys_rk4, 's--', color=COLORS["rk4"],
lw=1.2,
        markersize=4, label="Рунге-Кутта 4", zorder=3)

```

```

        ax.plot(xs, ys_adams, '^--', color=COLORS["adams"],
lw=1.2,
                markersize=4, label="Адамс (4-й порядок)",
zorder=2)

        ax.legend(facecolor=COLORS["bg"],
edgecolor=COLORS["grid"],
                labelcolor=COLORS["fg"], fontsize=10,
frameon=True)

        plt.tight_layout()
        plt.savefig(filename, dpi=150, bbox_inches="tight",
                facecolor=fig.get_facecolor())
        plt.close(fig)
        print(f" График сохранён: {filename}")

def run_session(eq_key, x0, y0, x_end, h, eps, output_dir,
session_id):
    eq      = EQUATIONS[eq_key]
    f       = eq["f"]
    exact   = eq["exact"]
    label    = eq["label"]

    print(f"\n{' '*65}")
    print(f"   ОДУ: {label}")
    print(f"   x ∈ [{x0}, {x_end}], h = {h}, ε = {eps}")
    print(f"   Точное решение: {eq['verify']}")
    print(f"{' '*65}\n")

    xs_e, ys_e = euler_solve(f, x0, y0, x_end, h)
    xs_rk, ys_rk = rk4_solve(f, x0, y0, x_end, h)
    xs_ad, ys_ad = adams_solve(f, x0, y0, x_end, h, eps)

    r_euler, _, _ = euler_runge_error(f, x0, y0, x_end, h,
eps)
    r_rk4, _, _ = rk4_runge_error(f, x0, y0, x_end, h,
eps)

    print("\n ТАБЛИЦА ЧИСЛЕННОГО РЕШЕНИЯ\n")
    print_table(xs_e, ys_e, ys_rk, ys_ad, exact)

    print_errors(xs_e, ys_e, ys_rk, ys_ad, exact, r_euler,
r_rk4, eps)

    fname = os.path.join(output_dir,
f"lab6_session{session_id}_eq{eq_key}.png")
    plot_results(xs_e, ys_e, ys_rk, ys_ad, exact,
                label, fname)

    return fname

def get_float(prompt, default=None):
    while True:
        s = input(prompt).strip()
        if s == "" and default is not None:
            return default
        try:
            return float(s)
        except ValueError:
            print(" Введите числовое значение!")

def get_int(prompt, choices):

```

```

while True:
    s = input(prompt).strip()
    try:
        v = int(s)
        if v in choices:
            return v
        print(f" Допустимые значения: {choices}")
    except ValueError:
        print(" Введите целое число!")

def get_yes_no(prompt):
    while True:
        s = input(prompt).strip().lower()
        if s in ("y", "д", "да", "yes", ""):
            return True
        if s in ("n", "н", "нет", "no"):
            return False

def main():

    OUTPUT_DIR =
os.path.join(os.path.dirname(os.path.abspath(__file__)),
"output")
    os.makedirs(OUTPUT_DIR, exist_ok=True)

    print(" ЧИСЛЕННОЕ РЕШЕНИЕ ОДУ")
    print("Методы: Усовершенств. Эйлера | РК4 | Адамс" )

    saved_plots = []
    session = 1

    while True:

        print("\n Выберите уравнение:")
        for k, eq in EQUATIONS.items():
            print(f"      [{k}] {eq['label']}")
        eq_key = get_int(" Номер уравнения: ",
list(EQUATIONS.keys()))
        eq = EQUATIONS[eq_key]

        print(f"\n Уравнение: {eq['label']}")
        print(f"      (По умолчанию: x0={eq['x0']},
y0={eq['y0']}, "
            f"x_end={eq['x_end']}, h={eq['h_def']})")
        x0 = get_float(f"      x0      [{eq['x0']}]: ",
eq['x0'])
        y0 = get_float(f"      y0      [{eq['y0']}]: ",
eq['y0'])
        x_end = get_float(f"      x_end [{eq['x_end']}]: ",
eq['x_end'])
        h = get_float(f"      h      [{eq['h_def']}]: ",
eq['h_def'])
        eps = get_float("      ε      [1e-4]: ", 1e-4)

        errs = validate_inputs(x0, x_end, h, eps)
        if errs:
            print("\n Некорректные данные:")
            for e in errs:
                print(e)
            print(" Попробуйте снова\n")
            continue

```

```

        try:
            fname = run_session(eq_key, x0, y0, x_end, h,
                                eps,
                                OUTPUT_DIR, session)
            saved_plots.append(fname)
            session += 1
        except Exception as ex:
            print(f"\n Ошибка при решении: {ex}")
            import traceback; traceback.print_exc()

        again = get_yes_no("\n Решить ещё раз с другими
данньми? (y/n) [y]: ")
        if not again:
            break

    print("\n" + "="*65)
    print(" Сохранённые графики:")
    for p in saved_plots:
        print(f" {p}")
    print("="*65)
    print(" Программа завершена.\n")

if __name__ == "__main__":
    main()

```

▪ Результаты программы (с графиками)

ЧИСЛЕННОЕ РЕШЕНИЕ ОДУ

Методы: Усовершенств. Эйлер | РК4 | Адамс

Выберите уравнение:

[1] $y' = -2y + 2x^2 + 2x$, $y(0)=1$

[2] $y' = y - x^2$, $y(0)=1$

[3] $y' = -y + \sin(x)$, $y(0)=0.5$

Номер уравнения: 2

Уравнение: $y' = y - x^2$, $y(0)=1$

(По умолчанию: $x_0=0.0$, $y_0=1.0$, $x_{\text{end}}=1.5$, $h=0.1$)

x_0 [0.0]:

y_0 [1.0]:

x_{end} [1.5]: 3

h [0.1]: 0.25

ε [1e-4]: 1e-3

=====

ОДУ: $y' = y - x^2$, $y(0)=1$

$x \in [0.0, 3.0]$, $h = 0.25$, $\varepsilon = 0.001$

Точное решение: $y = x^2 + 2x + 2 - e^x$

=====

ТАБЛИЦА ЧИСЛЕННОГО РЕШЕНИЯ

+-----+-----+-----+-----+-----+-----+

i	x _i	Эйлер улучш.	Рунге-Кутта 4	Адамс	Точное
0	0	1	1	1	
1	0.25	1.27344	1.27846	1.27846	1.27847
2	0.5	1.59058	1.60125	1.60125	1.60128
3	0.75	1.92855	1.94546	1.94546	1.9455
4	1	2.25806	2.28167	2.28162	2.28172
5	1.25	2.54158	2.57209	2.57196	2.57216
6	1.5	2.73101	2.76824	2.76797	2.76831
7	1.75	2.76474	2.80782	2.80735	2.8079
8	2	2.5638	2.61088	2.6101	2.61094
9	2.25	2.02706	2.07472	2.0735	2.07476
10	2.5	1.0249	1.06751	1.06565	1.06751
11	2.75	-0.608718	-0.580043	-0.582812	-0.580132
12	3	-3.08656	-3.08531	-3.08936	-3.08554

ОЦЕНКА ПОГРЕШНОСТИ

Усовершенствованный метод Эйлера (p=2):

Правило Рунге: $R = 1.21e-02$ (ϵ задана = $1.00e-03$)

Точность НЕ достигнута

Реальная погрешность: $4.77e-02$

Метод Рунге-Кутта 4-го порядка (p=4):

Правило Рунге: $R = 1.38e-05$ (ϵ задана = $1.00e-03$)

Точность достигнута

Реальная погрешность: $2.26e-04$

Метод Адамса (предиктор-корректор, p=4):

$\epsilon = \max|y_{\text{точн}} - y_i| = 3.83e-03$

Сравнение методов по точности:

Метод $\max|\delta|$

Усовершенствованный Эйлер $4.77e-02$

Рунге-Кутта 4 $2.26e-04$

График сохранён:

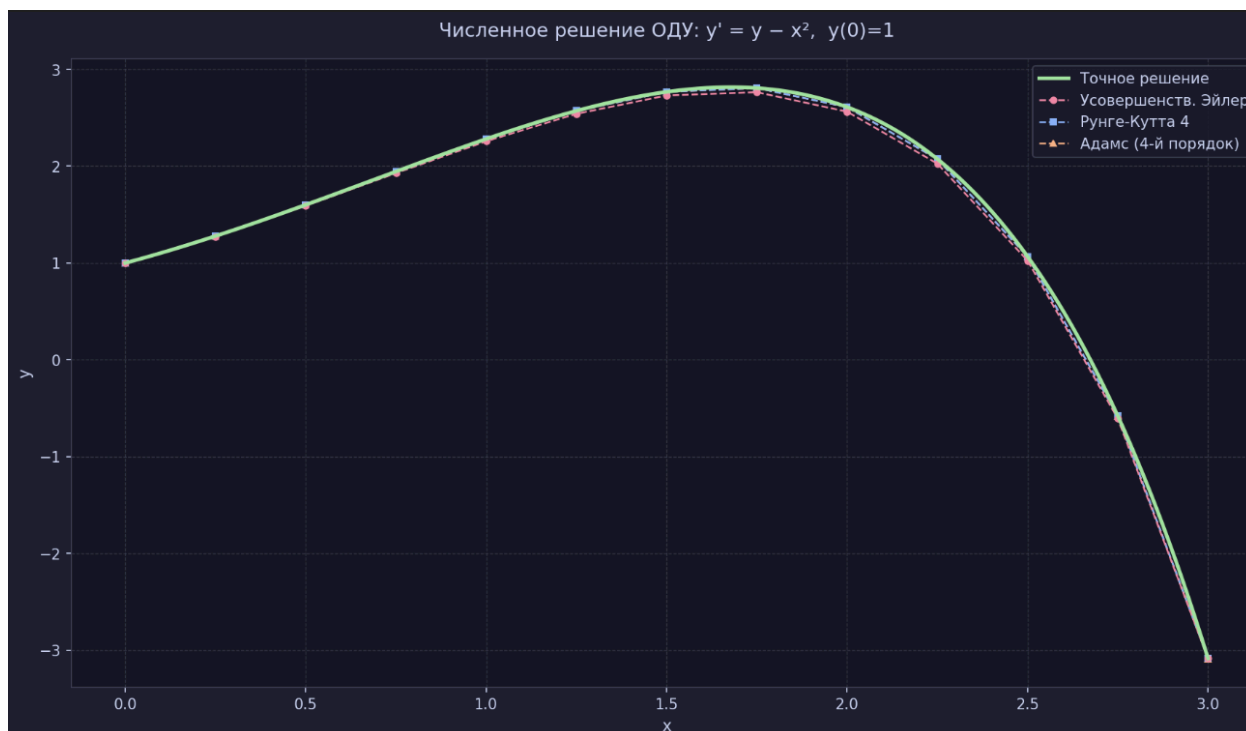
/Users/maria/PycharmProjects/CompMathLab6/output/lab6_session1_eq2.png

Решить ещё раз с другими данными? (y/n) [y]: n

Сохранённые графики:

/Users/maria/PycharmProjects/CompMathLab6/output/lab6_session1_eq2.png

Программа завершена.



Вывод

Выполнение лабораторной работы по вычислительной математике позволило мне познакомиться с численными методами решения обыкновенных дифференциальных уравнений. Данная работа дала мне возможность применить полученные теоретические знания на практике, развить навыки работы с информацией. В процессе выполнения я изучила и реализовала одношаговые методы: метод Эйлера, усовершенствованный метод Эйлера и метод Рунге-Кутты 4-го порядка, а также многошаговый метод Адамса (предиктор-корректор). Для оценки точности одношаговых методов использовалось правило Рунге, а для многошаговых методов — сравнение с точным аналитическим решением. Приобретённый опыт поможет мне при дальнейшем изучении предмета вычислительная математика.